

Salt Basin · Patch Notes

SALT BASIN NET WORKS · BOTTOM LINES WITH A RISING TIDE

SALT BASIN NET WORKS BUILD LOG

Patch Notes

What shipped to saltbasin.net, organized by release. Most recent first.

17 releases · build started June 3, 2026



CTA Visibility, Build Metrics & Deploy Process

LATEST

CTA buttons that link to draft or unpublished pages are now hidden from the public site — they only appear when the target page is live. All prior patch notes (v0.1–v0.16) are retroactively updated with quantified build metrics: director hours, Claude build time, and senior engineer equivalent hours. The post-deploy backlog script gains search and traceability capabilities. The full post-deploy process is documented for the first time.

♦ NEW — CTA VISIBILITY GATE

- `isLiveHref()` utility in `blocks/index.jsx`: returns false for internal links whose slug is not in the published live-page set, true for external links, anchors, and non-page routes (`/output/`, `/u/`, etc.)
- `liveSlugs` Set computed in `PublicSite.jsx` from all published pages with `status="live"`, passed to every `RenderSection`
- Six block types gated: `HeroBlock` (`cta1/cta2`), `TwoColBlock` (`cta1/cta2`), `CtaBlock` (`cta1`), `TimelineBlock` (`btns` array), `AppMockupBlock` (`cta1/cta2`), `ChoiceGridBlock` (per-choice `ctaLink`)
- Preview and admin modes receive `liveSlugs=null` → show all CTAs regardless of target page status (for content authoring)

♦ NEW — BUILD METRICS (RETROACTIVE, V0.1–V0.16)

- Every patch note entry now carries a metrics block: `directorHours`, `claudeBuildMins`, `engineerEquivHours`
- `directorHours`: time Betsy spent on architecture decisions, UX direction, requirements definition, and build supervision — billed at a principal architect/director rate
- `claudeBuildMins`: approximate wall-clock minutes Claude spent in active coding per release
- `engineerEquivHours`: senior engineer hours to produce the same output independently (includes their own arch/spec time since Claude works from director-provided direction)
- Computed at render time: $\text{directorCostUsd} = \text{directorHours} \times \text{DIRECTOR_RATE}$, $\text{leverageMultiple} = \text{engineerEquivHours} / (\text{directorHours} + \text{claudeBuildMins}/60)$
- Cumulative across v0.1–v0.16: ~57.5 director hours, ~3,465 Claude build minutes (~57.75h), ~1,010 engineer equivalent hours



◆ NEW — BACKLOG SCRIPT ENHANCEMENTS (ADD-V016-BACKLOG-ITEMS.MJS)

- `searchItems(items, query)`: fuzzy local search across title, summary, and `externalRef` — lets you find the right item to update/link without knowing the exact ref
- `findByVersion(items, version)`: returns all items whose `externalRef` starts with a given version string (e.g. "v0.15")
- Items created with `patchNoteVersion` tag: `tags` array includes "patch:vX.XX" for queryable version linkage
- Traceability fields: `parentRef` (links defects to their parent feature ref), `testScenarioRef` (links requirements to the scenario that validated them)
- Summary report now shows: updated / created deployed / created pending / skipped / not-found, grouped by capability

◆ NEW — POST-DEPLOY PROCESS DOCUMENTATION

- Post-deploy steps documented in `patchNotes.js` file header (canonical reference)
- Steps: (1) verify Render deploy on dashboard, (2) node scripts/add-vXXX-backlog-items.mjs, (3) node scripts/add-vXXX-test-scenarios.mjs if scenarios exist, (4) confirm `/output/patch-notes` renders the new release
- Why manual (not automated): scripts require `ADMIN_EMAIL + ADMIN_INITIAL_PASSWORD` in `.env` which are not in CI; idempotent but should only run after confirmed deploy; failed deploy should not create partial backlog records

◇ BEHIND THE SCENES

- No new DB tables or columns in this release
- `src/components/blocks/index.jsx`: `isLiveHref + NON_PAGE_PREFIXES` at top of file, `liveSlugs` param on `RenderSection` + all 6 block types
- `src/components/PublicSite.jsx`: `liveSlugs` Set computed from live pages after `currentPage` resolution, passed to each `RenderSection`
- `server/data/patchNotes.js`: metrics block added to all 16 prior entries + this entry; post-deploy steps in file header
- `scripts/add-v016-backlog-items.mjs`: `searchItems`, `findByVersion` helpers; `patchNoteVersion` tagging; traceability fields on all items



Lead Email Management & Auth Fix

Multi-email support lands on leads: each lead can now have personal and work addresses with per-email unsubscribe and primary designation. A critical login bug is fixed — admin access was broken after a lead with the admin's secondary email was converted to a member. Lead-to-member conversion now checks secondary email addresses too before blocking. The resume output page no longer spins forever when the founding page is in draft.

✓ FIXED

- Admin login restored: login() now tries the submitted password against ALL user records that match the email (primary users.email column + secondary verified user_emails rows). Prevents a member conversion from blocking an admin who had that email registered as a secondary address
- Lead conversion now checks user_emails secondary table before blocking with "email already registered" — gives a proper "sign in with X" message if it finds a match
- Resume output (/output/resume) no longer spins forever when the consulting-founder page is in draft — falls back to slug search and shows a clear error message instead

◆ NEW — MULTI-EMAIL PER LEAD

- lead_email_addresses table: each lead can have multiple contact emails, each with type (personal/work), org_name for work addresses, is_primary flag, and per-email subscribed toggle
- POST /api/leads: when an existing lead is matched by phone with a different email submitted, the new email is captured to lead_email_addresses automatically
- GET /api/leads/public/:publicId now includes contactEmails array
- POST /public/:publicId/contact-emails — add an additional email address
- PATCH /public/:publicId/contact-emails/:id — update type, org name, primary designation, or unsubscribe

◆ NEW — LEAD EMAIL UX

- LeadSuccessModal: when an existing lead is matched, shows the existing primary email and credential reminder so the lead knows they'll need their original password to access/convert
- LeadSuccessModal: when alternate email was submitted (phone match scenario), prompts to choose which should be primary — applies immediately without leaving the modal
- LeadView Email Addresses panel: shows primary email, all alternate addresses with type selector and org name for work, make-primary button, per-email unsubscribe toggle, and add-email form



◇ BEHIND THE SCENES

- lead_email_addresses table seeded in db.js bootstrap (idempotent CREATE TABLE IF NOT EXISTS)
- user_emails secondary check added to conversion route: SELECT from user_emails WHERE email = submitted_email AND verified = true
- Post-conversion: submitted email auto-registered in user_emails if different from users.email primary



Connections, Messaging & CRM Job Leads

Member-to-member social layer lands: connection requests, accept/decline flow, and direct messages gated to accepted connections. Each member gets a dedicated Inbox panel with sub-tabs for Messages, Requests, and Connections. The CRM gets manual lead creation (with skip-email option) and a new Job Lead type with company, hiring manager, job URL, status (New / Applied), and job description fields. The My Resume scope bug (admin seeing O sections) is fixed.

◆ NEW — MEMBER CONNECTIONS

- Connection request flow: "+ Connect" button on public profiles sends a pending request. Anonymous visitors see no button
- Inbox → Requests sub-tab: accept or decline incoming requests with real-time count badge
- Inbox → Connections sub-tab: grid of accepted connections with Profile link and Message shortcut
- Bidirectional connection lookup — only one row per pair (requester_id < recipient_id direction enforced at DB level)
- GET /api/members/me/connection-status/:slug — single call returns status, connectionId, direction, and targetUserId for the profile page

◆ NEW — DIRECT MESSAGING

- Members can message connections directly from a public profile (compose popover on "Message" button) or from Inbox → Connections
- Server enforces accepted connection before allowing any message — non-connections receive 403
- Inbox → Messages sub-tab: thread list with unread count badge; thread view with chat bubbles (gold = mine, grey = theirs), scroll-to-bottom on open, Enter to send / Shift+Enter for newline
- Read marking: GET /thread/:userId marks all their messages as read; unread count reflects state in real time
- GET /api/members/me/messages/unread-count — usable for future inbox badge in the nav

◆ NEW — INBOX PANEL

- New Inbox tab injected into admin_nav for both admin and member scopes via db.js bootstrap migration
- Three sub-tabs: Messages (thread list + thread view), Requests (pending incoming with count), Connections (accepted grid)
- Requests tab label shows live count: "Requests (2)" when 2 pending



◆ NEW — CRM ENHANCEMENTS

- Manual lead creation: "+ Add Lead" button in Leads panel header expands an inline form. Admin can create a lead without going through a public form
- "Skip confirmation email" checkbox suppresses the Brevo outbound email on manual creation
- Job Lead type (separate from Network leads): company, hiring manager info, job URL, status (New / Applied), and job description textarea
- Type filter tabs in Leads panel: All / Network / Job with live counts
- Job leads display company name prominently in the list and "Edit Job Info" inline form in the detail pane
- PATCH /api/leads/:id/job — update job-specific fields without touching the core lead record

✓ FIXED

- My Resume showed 0 sections for admin (Betsy): TAB_COMPONENTS entry was `() => <MyResumePanel />` — ignored all props including `scope`. Fixed to `(props) => <MyResumePanel {...props} />` so scope="admin" propagates and both admin + member site data load correctly
- MyResumePanel scroll: admin workspace has overflow:hidden; panel now wraps in flex:1/overflowY:auto container so full content is reachable

◇ BEHIND THE SCENES

- New tables: member_connections (UNIQUE(requester_id, recipient_id)), member_messages (sender_id, recipient_id, body, read_at)
- New columns on leads: lead_type, job_description, job_url, company, hiring_manager, job_status
- All migrations are idempotent ALTER TABLE IF NOT EXISTS / CREATE TABLE IF NOT EXISTS in db.js bootstrap
- scripts/add-v015-backlog-items.mjs — idempotent backlog reconciliation for v0.15 items
- scripts/add-v015-test-scenarios.mjs — 18 test scenarios covering every v0.15 feature plus regression paths



Platform Merge — HERQ, NRM, Services & Governance

The largest single release in the build. Five product lines merge into the unified Salt Basin platform: HERQ (HR analytics layer), NRM (network relationship manager), Services Catalog, Standards Library, and Governance. Analytics, Data Lineage, and Security hardening round it out. The My Resume tab ships to the admin shell with layout preset builder, AI-tailored preset generation, and PDF preview. Fifty-three backlog items written or updated across twelve capability groups.

◆ NEW — PLATFORM PRODUCTS

- HERQ: compensation benchmark widgets, headcount heatmap, retention-risk matrix, org-chart renderer — all reading from the analytics layer
- NRM: network health dashboard, relationship map, engagement tracker, pipeline view by relationship tier
- Services Catalog: browsable card grid of consulting offerings, each with scope, deliverables, and rate band
- Standards Library: versioned policy and process documents with section-level approval workflows
- Governance: board/committee charter management, meeting minutes, decision log, and policy attestation tracker

◆ NEW — ANALYTICS & LINEAGE

- Analytics capability group seeded (12 backlog items): event pipeline, dashboard renderer, KPI store, and threshold alerting
- Data Lineage capability group (8 items): column-level lineage graph, impact analysis, compliance tagging, and audit export
- Security hardening: rate limiter on login/reset, cookie attribute fix on logout, lazy session purge, org-invite email enumeration fix, 16-char lead access passwords



✦ NEW — MY RESUME (ADMIN SCOPE)

- My Resume tab in admin shell ("My Profile" view) — same panel as member scope but loading both admin CMS sections and member sections together
- Resume preset builder with layout template picker (Classic SB, Modern SB, Minimal, Executive) and per-section include toggles with drag-to-reorder
- AI interpreter: paste a job description → Claude returns tailored summary, highlighted skills, rewritten bullets, and gap analysis
- Accept-and-save flow: agent output becomes a new named preset; discard clears it without touching existing presets
- PDF preview iframe inline in the panel
- Primary preset designation controls which resume `/output/resume` serves publicly

✦ NEW — ACTION BUTTON EDITOR

- Every section in the editor now shows a Section Actions card — add CTA buttons with a guided link picker that lists site pages, anchors, and `/output/*` routes
- Button style picker: primary / secondary / ghost / danger
- Saved to `fieldMeta.actions` per section; rendered in public view and preview

◇ BEHIND THE SCENES

- `scripts/add-v014-backlog-items.mjs` — idempotent backlog reconciliation for all v0.14 items
- 53 new/updated backlog items, 12 capability groups touched
- `admin_nav` Inbox tab injection in `db.js` bootstrap
- My Resume scoped via AdminShell `TAB_COMPONENTS` (bug surface for v0.15 fix)



Resume, Templates & Profile Depth

Major expansion of member profile capabilities: categorized section template modal, career resume page seeded by default, "My Resume" member app with preset builder and AI interpreter, expanded case study + client snapshot blocks, Skills + Industry Wheel + Career Timeline templates, configurable section action buttons, and a generic member JSON store for future extensibility.

◆ NEW — TEMPLATE CATEGORIES

- Section template modal rebuilt: flat grid replaced by a 4-category sidebar (General, Member Profile, Org Profile, Project Management)
- General: Blank, Hero, Two Column, CTA, Stat Grid
- Member Profile: Career Timeline, Case Studies, Client Snapshot, Skills, Technology, Industry Wheel, Executive Summary, Services/Offerings, Resume Block
- Org Profile: Company Timeline, Roadmap, Product Mockup, Team/Leaderboard
- Project Management: KPI Dashboard, Status Heatmap, Project Timeline, Feature Cards, Decision Tree, Output Generator
- Each category has a tab count and its templates include updated default field seeds tailored to their use case

◆ NEW — MY RESUME

- New "My Resume" tab in the member admin shell (next to My Profile)
- Resume preset builder: create named presets by checking which pages/sections to include in a given resume variant
- Primary preset designation — the primary resume is the publicly downloadable version on the member's profile page
- Public download link displayed in the panel once a primary preset is configured
- Resume interpreter agent: paste any job description and the AI returns a tailored professional summary, top skills to highlight, experience bullet rewrites, and gap analysis
- Copy-to-clipboard and direct link to the resume output renderer

◆ NEW — RESUME PAGE DEFAULT

- New member sites are seeded with a "Resume" page out of the box (in addition to Home, About, Contact)
- The Resume page includes: Resume Hero section (navy, with Download PDF + Contact CTA), Career Timeline, Role Detail (dynamic roles list), and Skills
- All role and skill entries are fully editable from the member admin



◆ NEW — BLOCK & EDITOR IMPROVEMENTS

- Skills block: skill groups with category headings, name, proficiency level (expert/proficient/familiar), and years — rendered as colored chips
- Client Snapshot block: per-client inputs for industry, employer sponsor, capabilities delivered/touched, technology delivered/touched, revenue range, and tags; rollup view groups by industry, capability, or revenue
- Case Studies block: expanded to 8 structured fields per case (clientSummary, problemStatement, kpilImprovement, methodsTaken, challenges, impact, feedback, tags) with accordion expand in both editor and public view
- Career Timeline configurable actions — no longer hardcoded to "View Resume"; CTA buttons set via the Section Actions editor
- Section Actions card visible for every section in the editor — add, edit, and remove CTA buttons (label, href, style)
- Photo/media upload support in any section via + Add Photo Field in the editor Content Fields card

◇ BEHIND THE SCENES

- New table: member_json_store (user_id, key, data, updated_at) — generic key-value store for member settings that don't fit the draft/published content model
- New API: GET/PUT /api/members/me/resume-presets persists preset JSON to member_json_store key="resume_presets"
- New component: src/components/admin/MyResumePanel.jsx
- defaultMemberSite.js updated: Resume page added as 3rd page with 4 sections (hero, timeline, roles, skills)
- SectionTemplateModal.jsx fully rewritten: 37 template definitions across 4 categories, category sidebar, Step 1 grid auto-sizes to category
- AdminShell: "My Resume" tab wired for isMember; resume excluded from PublishBar



Field Settings, New Block Library, and Section Templates

Every content field in every section now has a full settings panel: visible/hidden, auditable edit history, field type (13 options), predefined value sets for select/multi-select, and cascading/dependent field rules. Nine new block types ship — drawn directly from the PPTX visual references — giving the CMS the ability to produce KPI dashboards, roadmaps, status heatmaps, leaderboards, executive summaries, app mockups, choice grids, interactive decision trees, and an AI-powered drag-and-drop output generator. Section creation is now a two-step visual modal: pick a template from a card grid, then name and configure it. The sidebar section list scrolls independently so the "+ Add Section" button is always pinned and visible.

◆ NEW — FIELD SETTINGS SYSTEM

- Every field in every section now has a ⚙ Settings tab: visible/hidden toggle, auditable toggle (captures before/after edit history), field type selector (text, textarea, number, date, boolean, select, multiselect, URL, email, JSON, image, color, richtext), predefined value set editor for select/multiselect types, description/hint field
- Source tab (existing source type system, now tab-organized) — user_input / merged / derived / direct with merge path and capability tags
- Cascade tab — define rules so selecting a value in one field filters the allowed options of a downstream field (dependent dropdowns)
- Clone (📄) and Remove (X) controls on every field row
- + Add Field button at the bottom of each section to create new custom fields inline
- field_audit_log table + POST/GET /api/field-audit route — any field with auditable: true silently logs user + before/after value on every save



◆ NEW — BLOCK TYPES (9)

- KPI Dashboard — pastel metric panels grid; each panel has value, change indicator, caption, and custom color
- Roadmap — horizontally scrollable timeline with gradient-colored milestone nodes and status pills (complete / in-progress / planned / blocked)
- Status Heatmap — color-coded row × column status matrix (green/yellow/red/blue/teal cells) with auto-generated legend
- Leaderboard — ranked list with gold/silver/bronze highlights, optional avatars, value and change delta per entry
- Executive Summary — two-column layout with accent-colored stat cards on the left and a contact/info card on the right
- App Mockup — phone / tablet / browser frame with gradient fill and screen content or images; supports multi-screen side-by-side
- Choice Grid — interactive colored option tiles; click a tile to expand it with a CTA link
- Decision Tree — interactive YES/NO branching flowchart; click branches to navigate, Reset button clears state
- Output Generator — drag-to-reorder content block palette, output type selector (resume / executive bio / cover letter / LinkedIn summary / one-pager / proposal intro), audience/job-description textarea, AI-powered Generate button that calls the member agent endpoint

◆ NEW — SECTION TEMPLATE MODAL

- Replacing the old plain-text section form: "+ Add Section" opens a 2-step visual modal
- Step 1: card grid of 15 pre-built templates (Blank, Hero, KPI Dashboard, Roadmap, Executive Summary, Heatmap, Leaderboard, App Mockup, Choice Grid, Decision Tree, Output Generator, Cards, Two Column, Timeline, Stat Grid, CTA) — each with icon, description, and default column count
- Step 2: name the section, set columns (1–4), choose background, set visibility — then seed fields are pre-filled from the template

✓ FIXED

- Sidebar section list now scrolls independently — the "+ Add Section" button is pinned to the bottom of the sidebar and is always visible regardless of how many sections exist
- Pages list and section label stay fixed; only the section rows scroll



◇ BEHIND THE SCENES

- New table: field_audit_log (user_id, section_id, field_key, before_value, after_value, created_at)
- New route file: server/routes/fieldAudit.js — POST logs an audit entry, GET retrieves history by sectionId and optional fieldKey
- New component: src/components/admin/SectionTemplateModal.jsx (15 templates, 2-step flow)
- blocks/index.jsx grew from 3649 → ~5400 lines with 9 new block components + STATUS_PILL helper
- REGISTRY expanded from 26 → 35 block types; BLOCK_TYPES export auto-derives from REGISTRY so no separate update needed
- addSection() in AdminShell now accepts columns + pre-seeded fields from the template instead of the old hardcoded type-switch logic



A real member sign-up process

The end-to-end sign-up loop a real human would use, end to end. A lead viewing their own record can convert to a member in one click (with a password re-entry confirmation) and lands in the member dashboard auto-signed-in. The Sign In button on the public site goes to a generic /login page instead of /admin/login. That page now also lets people reset a forgotten password and recover the email they signed up with. All of these flows have reCAPTCHA v3 wiring in place — the code is a no-op until you register a site key and paste the secret into Render env, then captcha kicks in everywhere at once.

◆ NEW

- POST /api/leads/public/:publicId/convert — authenticated lead promotes themselves to a member without re-entering anything but a password confirmation. Lead password_hash is reused as the new user password_hash (no re-hash). Lead row persists with converted_user_id set so all prior context (emails, activity, merged-from history) stays queryable
- LeadView "I know you love it — go ahead and convert to member" CTA card with a password-confirm modal
- /login as the canonical generic sign-in URL. /admin/login still works as a back-compat alias
- Forgot password? flow on the sign-in page → enter email → email with a single-use 1-hour reset link → /reset/:token page with new password + confirmation
- Forgot your email? flow on the sign-in page → enter phone → email gets sent to the address on the lead with that phone reminding them which email to use
- reCAPTCHA v3 wired into signup, lead→member convert, password reset request, and email recovery. Skipped at runtime when RECAPTCHA_SECRET_KEY isn't set, so safe to ship before keys are registered

◇ CHANGED

- Sign-in page redirects post-login based on role: admin → /admin, member → /member. A ?next= URL param (same-origin only) overrides this
- Sign-in page title is "Sign In" (not "Admin Login") since members use the same page
- Password reset deletes all existing sessions for the user as a precaution — anyone signed in elsewhere will need to re-sign-in with the new password



◇ BEHIND THE SCENES

- New password_reset_tokens table: token (random 32-byte hex), user_id, expires_at, used_at, created_at. Used tokens for the same user are bulk-invalidated when one is consumed (defensive against concurrent requests)
- Email recovery walks leads.phone → leads.converted_user_id → users.email; phone is normalized to digits-only to match the existing lead-capture convention
- Both recovery endpoints always return 200 regardless of whether a record matched, to defeat email + phone enumeration
- reCAPTCHA scaffolding (server/lib/recaptcha.js + src/lib/recaptcha.js) reads env vars: VITE_RECAPTCHA_SITE_KEY (Netlify, frontend), RECAPTCHA_SECRET_KEY + optional RECAPTCHA_MIN_SCORE (Render, backend). To turn it on: register a v3 site at <https://www.google.com/recaptcha/admin>, paste both keys into the respective dashboards, redeploy
- reCAPTCHA NOT yet applied to the lead-capture endpoint — needs each of the public lead-capture form blocks (Join the Network, For Companies, contact, assessments) to send a token first. Tracked as a follow-up on task #28



Pre-onboarding fixes — My Profile and dynamic resume

Targeted fixes ahead of inviting members. The primary editing surface is "My Profile" again — the v0.9 nav refactor obscured the use case with the generic label "Content / My Site". The Config tab opens with an explanation card so newcomers know what each section configures. Resume now uses a dynamic add/remove list — members can have as many roles as their careers need, with start/end dates and a "Current" toggle.

✓ FIXED

- "Content" / "My Site" → "My Profile" everywhere: seeded nav, hardcoded member tab strip, and a one-shot migration that relabels the existing `config_state.admin_nav` row in production without touching any manual nav edits
- Config tab top of panel: new intro card explains what Config is for (member: "how your profile looks and where it links to"; admin: "platform-level saltbasin.net configuration"), distinguishing it from My Profile (content)

◆ NEW

- Resume roles: dynamic add/remove list. Each role has title, company, start date, end date or "Current", and a description. Up/down to reorder, × to delete. No more 2-slot limit
- `scripts/add-tonight-defect-items.mjs` records the five bugs flagged tonight as kind="defect" backlog items with full requirement detail and acceptance criteria — three completed, two pending follow-up

◇ BEHIND THE SCENES

- ResumeBlock renderer accepts both shapes: new `f.roles` array (preferred) and legacy `f.role1/f.role1Desc` keys (fallback) so existing member data keeps rendering until next edit
- EditorPane gained array-aware field detection: when a section field is an array (today: roles), a dedicated list editor renders instead of the default string input. Pattern generalizes to domains and service cards in the next pass



QA System, Audit Log, and a Data-Driven Admin

The admin grows up. A new QA tab lets the admin author test scenarios for every deployed feature, run them step-by-step, and have failed steps auto-create defect backlog items linked back to the failing scenario. Every mutation across the QA and backlog routes lands in a new `audit_events` log so nothing gets changed silently. The admin tabs themselves reorganize into "Platform Lifecycle Management" and "Customer Relationship Management" views — and the whole nav structure now lives in `config_state` rather than hardcoded JSX, so future relabels and reorderings happen without a code push.

◆ NEW

- QA tab (under Platform Lifecycle Management): author test scenarios tied to deployed backlog items, with ordered steps capturing action + expected outcome
- Log Test Run modal: per-step pass / fail / blocked verdicts with notes and evidence URLs, environment selector (test vs prod), overall notes
- Auto-defect creation: any failed step spawns a `kind="defect"` backlog item parented to the scenario's primary feature, with the failing step's evidence carried into the defect summary
- Multi-select linked features on a scenario — one scenario can cover multiple deployed features, with one marked as "primary" (the parent for auto-defects)
- Audit log: every create / update / delete on QA and backlog routes writes a row in `audit_events` with before/after JSONB, the source (`manual_ui` / `brain_dump` / `bulk_script` / `jira_sync` / `seed`), and the actor
- Admin nav reorganized into views: Content, Platform Lifecycle Management (Backlog + QA), Customer Relationship Management (Leads + Net Works), and System (Config)

◇ CHANGED

- Admin nav structure is now data-driven from `config_state["admin_nav"]` — labels, grouping, order, and presence can change without touching code (the component identities stay in a small in-code registry, since React components can't be loaded from JSON)
- Native `<option>` dropdowns are now legible on dark OS themes (global color rule in `brand.css`)



◇ BEHIND THE SCENES

- New tables: test_scenarios, test_scenario_steps, test_runs, test_run_step_results, test_scenario_features (junction), audit_events
- backlog_items gets a test_scenario_id column for back-navigation from a defect to the scenario that surfaced it
- Optimistic concurrency on PATCH /api/qa/scenarios via expectedUpdatedAt: writes can't silently overwrite changes that happened between read and apply
- Default admin_nav structure seeded into config_state at bootstrap if missing; deleting that row and rebooting re-seeds it



JIRA + Templates + Scrum Agent + Admin Polish

Three new tracks land in parallel: JIRA Cloud read-only import wired up; three curated member templates seeded; the Scrum Agent goes live as a chat panel docked inside the Backlog tab (Phase A scaffold — Claude-powered, no tools yet). Plus a focused round of admin UX polish.

◆ NEW

- JIRA Integration (Phase A · read-only pull): Config card → save credentials → Test → Import. Issues land in a new "JIRA Mirror" capability group with status/priority/type mapped
- Member Templates Phase A: Operator Profile / Consulting Practice / Coach-Speaker starters seeded (gallery UX next session)
- Scrum Agent Phase A: floating ◆ button in Backlog opens a docked chat panel. Real Claude responses with conversation persistence in localStorage. Propose-and-approve tools wire in Session 3
- Salt Basin admin brand palette card (with Reset to defaults). Edits flow through publicConfig → saltbasin.net recolors
- Date pickers for date-shaped resume fields (start / end / since / until / etc.) — was free-text

🔄 CHANGED

- Split view is now drag-resizable with a gold-tinted divider. Double-click resets to 55/45
- Persisted split ratio per browser, separately for admin vs member scope

✓ FIXED

- Brand colors were invisible to admin — the card was wrapped in `{isMember && ...}`. Now visible (and functional) for both scopes
- TDZ crash in AdminShell: split-view useEffect depended on `sidebarOpen` declared further down → blank admin page on first load. Hook order reordered with inline comment to prevent regression

◇ BEHIND THE SCENES

- New schema: jira_config, member_templates, agent_threads, agent_messages, plus brand_kit + tech_stack columns on existing tables
- ANTHROPIC_API_KEY pushed to Render env so Scrum Agent calls Claude directly



Backlog Enhancements + 3 Outputs

Per-person split (Betsy hours vs Claude hours, Betsy activities vs Claude activities) replaces the simple work-split slider. Live capability-group rollups. New Outputs catalog inside the Backlog tab linking to three print-friendly reports including a pre-AI cost comparison.

✦ NEW

- Per-person split: hours_betsy / hours_claude / activities_betsy / activities_claude (replaces single work-split %)
- Live capability rollups — switching groups in the left rail re-aggregates the metric chips for that scope
- Outputs catalog inside Backlog → Outputs sub-tab
- Tech Stack Architecture output (5-layer architecture diagram + per-capability tech mapping)
- Marketing Product One-Pager output (Salt Basin Net Works positioning, three pillars, capability inventory)
- Build Progress Report output (now with pre-AI cost comparison @ \$150/hr × 2.5× effort multiplier + delivery timeline)

🔄 CHANGED

- BacklogDrawer replaces single work-split slider + time-minutes input with a 2×2 PersonInput grid

Admin Backlog

A real product-management dashboard inside the admin. Every feature shipped becomes a backlog item with full user story / requirement detail / business rules / design spec / acceptance criteria. Tier-cost workarounds documented as their own card type.

✦ NEW

- Admin Backlog tab (admin-only): capability groups left rail, requirement cards right
- 35 requirements seeded covering all work to date, organized into 12 capability groups
- 6 tier workarounds documented (Resend → Brevo, Zoho Free, Render Free + UptimeRobot, Netlify credit lock, Supabase Free, GitHub Actions Free)
- Side drawer for each requirement: fully editable user story / requirement / rules / spec / criteria / process steps
- Tech stack per capability + cost estimates per requirement



Email Infrastructure

Outbound email goes from console-stubbed to fully delivered. Custom-domain mailbox at betsysalter@saltbasin.net (Zoho Free). Lead confirmations + Betsy alerts on every submission. Switched provider mid-build when Resend collided with Wix DNS limits.

✦ NEW

- Zoho Mail Forever Free Plan for betsysalter@saltbasin.net
- Lead confirmation emails (URL + password) sent on every submission
- Betsy alert emails on every new lead AND repeat activity
- Admin Config panel: Outbound Email Identity + New-Lead Notifications cards
- Inline "Send test email" probe button

🔄 CHANGED

- Email provider: Resend → Brevo. Wix DNS does not allow subdomain MX records (Resend requires send. <domain>); Brevo uses CNAME-based DKIM at the apex — no subdomain MX needed
- `config.email.fromAddress` now reads from the published config (admin-editable)

Multi-Tenant CMS

Every member operator gets the same multi-page admin Betsy uses, scoped to their own data. Brand colors per member. Net Works opt-in banner on the Salt Basin home page that surfaces opted-in operators automatically.

✦ NEW

- AdminShell takes a ``scope`` prop — same shell powers admin + every member
- Member sites have their own draft + published JSON in ``member_sites``
- Member Config: brand colors, Net Works opt-in (logo + blurb + company name), BYO Anthropic API key slot
- Default 3-page starter site (Home / About / Contact) seeded on first member visit
- NetWorksBanner block on Salt Basin home page — horizontal-scroll cards of opted-in members
- Salt Basin admin gets a Net Works tab listing every signed-up member



Lead Capture & Outputs

Forms become real leads. Each lead gets a private password-protected URL. The first round of print-friendly output pages ships (resume, proposal, case study, one-pager) — member-gated with teaser previews for non-members.

◆ NEW

- Lead capture from every public form with match-and-merge dedup on email + phone
- Per-lead private record at /lead/<publicId> behind a bcrypt password
- Lead-to-member conversion path (pre-fill signup with lead context)
- Print-friendly Resume / Case Study / Proposal / One-Pager output pages
- Auth gate: non-members see teaser, members see full content + Print/PDF
- References request flow (separate lead source, dedicated form)
- Data Notice page + inline disclaimers on every form

◇ CHANGED

- Resume output now includes Industries Served + client snapshots + tech stack
- Tech stack edits: NetSuite + Zone Billing + Avalara + Adaptive + Informatica moved to Integration Design; TIBCO BW moved to Adjacent Exposure



VO.2 · JUNE 4, 2026

Founder-First Public Site

The public-facing Salt Basin home page comes together. Hero rebuilt around the founder + mission split, the Industries × Domains wheel ships, services and case studies land, mobile responsiveness throughout.

◆ NEW

- About / Intro hero: face-of-the-founder + Salt Basin mission split
- Industries × Domains interactive SVG wheel with per-industry dashboards
- Niche Solutions panel with CPQ / Billing / Integration tabs
- Timeline block (7 roles, click-to-expand)
- Case Studies block (3 detailed SAO studies)
- Technology Experience block (Hands-On / Integration / Adjacent groupings)
- Join the Network + For Companies CTA sections
- Mobile responsive layout, hamburger nav
- Cold-start branded loader (Render free-tier UX)

VO.1 · JUNE 3, 2026

Foundation

First commits land. The Salt Basin Net Works CMS goes from idea to a working two-state (draft / published) admin scoped behind a bcrypt-hashed login, served from an Express + Postgres backend on Render and a React + Vite frontend on Netlify.

◆ NEW

- Multi-page CMS with draft / publish workflow
- Block library: hero, cards, two-column, contact, scripture, text, CTA
- Strategic Operator brand tokens (navy / gold / cream palette, Cormorant + Inter type)
- Password-protected pre-launch landing gate
- Admin sidebar + editor + live preview (split-view default)

Generated from the curated release log at `server/data/patchNotes.js`. Future releases append to that file and auto-publish here.

